

BITÁCORA DE EJECUCIÓN Y CIERRE — SPRINT 3

Foco del Incremento: Búsqueda y Disponibilidad — Búsqueda por ciudad y fechas, Calendario de disponibilidad, Favoritos, Políticas, Reseñas y Compartir en redes **Stack Tecnológico:** Java 17 / Spring Boot 3.5 / Spring Security 6 / MariaDB / React 19 / Vite / SpringDoc OpenAPI

1. Resumen del Incremento (Scope)

El Sprint 3 se centró en dotar a la plataforma de funcionalidades de búsqueda avanzada, visualización de disponibilidad, gestión de favoritos, y contenido social. Se implementó un motor de búsqueda unificado con filtros dinámicos (ciudad, fechas, capacidad, precio, categoría) utilizando Specifications de Spring Data JPA, un sistema de reservas con control de concurrencia mediante optimistic locking, un calendario de disponibilidad visual, favoritos con toggle desde las cards del home, políticas de producto, reseñas con puntuación de estrellas, y la capacidad de compartir alojamientos en redes sociales. Adicionalmente, se incorporó documentación automática de la API mediante SpringDoc OpenAPI con Swagger UI.

2. Arquitectura del Sistema e Integración

2.1. Backend (Spring Boot + Spring Security 6)

Se expandió la arquitectura base con nuevos módulos y se refinaron los existentes:

```
Controller → Service (Interface + Impl) → Repository → Entity / DTO
```

Matriz de Componentes Introducidos:

Módulo	Entidad (Entity)	Objetos de Transferencia (DTO)	Capa de Servicio	Capa de Control (Controller)
Reservas	Reservation	CreateReservationRequest, ReservationResponse, AvailabilityResponse, OccupiedRange	ReservationService	ReservationController
Favoritos	Relación en User	—	UserService extendido	FavoriteController
Políticas	Policy	PolicyDTO	PolicyService	PolicyController
Reseñas	Rating	RatingDTO, RatingRequest	RatingService	RatingController
Subida imágenes	—	UploadResult	CloudinaryService	UploadController

- **Estrategia de Búsqueda:** Se reemplazó el enfoque de `@Query JPQL` por **Specifications** (Criteria API de Spring Data JPA). Esto permite componer filtros dinámicamente sin escribir strings SQL, manteniendo el tipado fuerte del compilador y la consistencia con el patrón del proyecto.
- **Control de Concurrencia:** Se implementó `@Version` (optimistic locking) en las entidades `Reservation` y `Lodging` para prevenir la doble reserva del mismo alojamiento en fechas solapadas. Si dos usuarios intentan reservar simultáneamente, el segundo recibe un HTTP 409 Conflict.
- **Búsqueda con Specifications:** La lógica de filtrado se construye en el servicio encadenando `Specification.and()` :
 - Filtro por ciudad (LIKE case-insensitive)
 - Filtro por capacidad de huéspedes
 - Filtro por categoría
 - Filtro por rango de precio

- Exclusión de alojamientos con reservas solapadas (filtrado en Java post-query)
- **Autocompletado de Ciudades:** Endpoint `GET /api/lodgings/cities?q=` que devuelve ciudades distinct con coincidencia parcial. El frontend implementa debounce de 300ms.

2.2. Frontend (React + Vite)

Evolución de la SPA con nuevas páginas y componentes extraídos:

```
src/
├── components/
│   ├── Reservation/
│   │   └── ReservationModal.jsx    (Modal de confirmación de reserva)
│   ├── ReviewsSection/
│   │   ├── ReviewsSection.jsx    (Sección de reseñas con estrellas)
│   │   └── ReviewsSection.css
│   └── ShareModal/
│       ├── ShareModal.jsx        (Pop-up para compartir en redes)
│       └── ShareModal.css
├── pages/
│   ├── Home/Home.jsx            (Buscador con autocompletado + fechas)
│   ├── SearchResults/
│   │   ├── SearchResults.jsx    (Página de resultados con filtros)
│   │   └── SearchResults.css
│   ├── ProductDetail/
│   │   └── ProductDetail.jsx    (Calendario, precio, reseñas, políticas)
│   ├── Favorites/
│   │   ├── FavoritesPage.jsx    (Lista de favoritos del usuario)
│   │   └── FavoritesPage.css
│   └── Admin/
│       └── AdminPolicies.jsx    (CRUD de políticas en panel admin)
├── hooks/
│   └── useAuth.js                (Wrapper del contexto de autenticación)
├── context/
│   └── AuthContext.jsx          (Lazy initialization del estado de sesión)
└── services/
    └── api.js                    (Interceptor Bearer + manejo de 401)
```

- **Extracción de Componentes:** La sección de reseñas se extrajo a `ReviewsSection`, reduciendo `ProductDetail.jsx` de 356 a 242 líneas.
- **Búsqueda con Transición (React 19):** Se utilizó `useTransition` para evitar warnings de `setState` síncrono en efectos, siguiendo las recomendaciones de React 19.
- **Calendario:** Se integró `react-datepicker` para la selección de fechas con fechas ocupadas deshabilitadas.
- **Fallback Visual:** Todas las imágenes tienen `onError` para mostrar placeholder si falla la carga, y `loading="lazy"` para rendimiento.

3. Trazabilidad de Historias de Usuario (User Stories)

ID	Historia de Usuario	Componente / Vista UI	Endpoint Backend	Criterio de Aceptación / Estado
US #22	Realizar búsqueda por ciudad y fechas.	Home.jsx, SearchResults.jsx	GET /api/lodgings/search, GET /api/lodgings/cities	Búsqueda unificada con filtros. Autocompletado de ciudades. Resultados con cards.
US #23	Visualizar disponibilidad en calendario.	ProductDetail.jsx	GET /api/lodgings/{id}/availability, POST /api/reservations	Calendario doble con fechas ocupadas deshabilitadas. Reserva con confirmación.
US #24	Marcar producto como favorito.	ProductCard.jsx	POST /api/favorites/{lodgingId}	Corazón visible solo para autenticados. Toggle con un clic.
US #25	Listar productos favoritos.	FavoritesPage.jsx	GET /api/favorites, DELETE /api/favorites/{lodgingId}	Lista de favoritos con opción de quitar.
US #26	Ver bloque de políticas del producto.	ProductDetail.jsx	Propiedad lodging.policies	Título subrayado. Políticas en columnas con ícono + título + descripción.
US #27	Compartir producto en redes sociales.	ShareModal.jsx	N/A (Frontend)	Pop-up con opciones Facebook, Twitter, WhatsApp. Imagen + descripción + enlace.
US #28	Puntuar producto con estrellas.	ReviewsSection.jsx	POST /api/ratings, GET /api/ratings/lodging/{id}	Sistema de 1-5 estrellas. Promedio dinámico. Reseñas con nombre, fecha, comentario.
US #29	Eliminar categoría con confirmación.	AdminCategories.jsx	DELETE /api/categories/{id}	ConfirmDialog reemplazó window.confirm.

4. Catálogo de Endpoints Nuevos

4.1. Búsqueda y Disponibilidad

Método	Endpoint	Acceso (RBAC)	Descripción
GET	/api/lodgings/search	Público	Búsqueda unificada con filtros
GET	/api/lodgings/cities	Público	Autocompletado de ciudades
GET	/api/lodgings/{id}/availability	Público	Disponibilidad por rango de fechas
POST	/api/reservations	Autenticado	Crear reserva
GET	/api/reservations/{id}	Autenticado	Detalle de reserva

4.2. Favoritos

Método	Endpoint	Acceso (RBAC)	Descripción
POST	/api/favorites/{lodgingId}	Autenticado	Agregar favorito
DELETE	/api/favorites/{lodgingId}	Autenticado	Quitar favorito
GET	/api/favorites	Autenticado	Listar favoritos

4.3. Políticas

Método	Endpoint	Acceso (RBAC)	Descripción
POST	/api/policies	ADMIN	Crear política
PUT	/api/policies/{id}	ADMIN	Actualizar política
DELETE	/api/policies/{id}	ADMIN	Eliminar política
GET	/api/policies	Público	Listar políticas
GET	/api/policies/{id}	Público	Política por ID

4.4. Reseñas

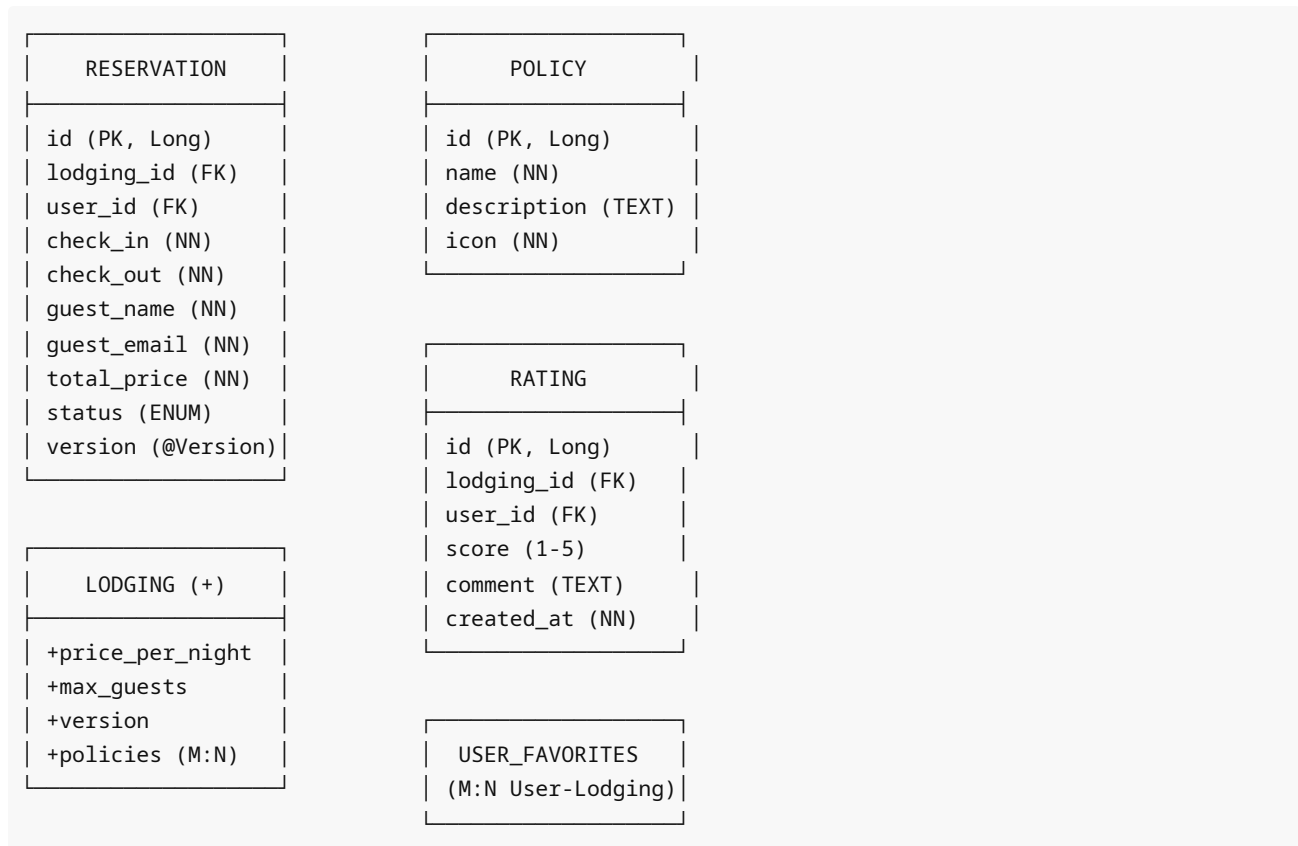
Método	Endpoint	Acceso (RBAC)	Descripción
POST	/api/ratings	Autenticado	Crear reseña
GET	/api/ratings/lodging/{lodgingId}	Público	Reseñas por alojamiento

4.5. Subida de Imágenes

Método	Endpoint	Acceso (RBAC)	Descripción
POST	/api/upload	ADMIN	Subir imagen a Cloudinary

5. Modelo de Datos

Nuevas Entidades



- **Reservation:** Nueva entidad central para el manejo de disponibilidad. Usa `@Version` para optimistic locking. FK a Lodging y User. El total se calcula como `días × pricePerNight`.
- **Policy:** Similar al patrón de Feature (catálogo reutilizable). Relación M:N con Lodging vía `lodging_policies`.
- **Rating:** Almacena reseñas con puntuación de 1 a 5. Relacionada con Lodging y User.
- **User:** Se agregó relación `@ManyToMany` con Lodging para favoritos vía `user_favorites`.
- **Lodging:** Se agregaron `pricePerNight`, `maxGuests`, y `@Version`. Nueva relación M:N con Policy.

6. Decisiones Técnicas Clave

- **Specifications vs @Query:** Se eligió Specifications (Criteria API) sobre `@Query JPQL` para mantener la consistencia con el patrón del proyecto (solo derived queries). Specifications permite componer filtros dinámicamente con tipado fuerte, aunque la exclusión de fechas ocupadas se realiza en Java post-query por no requerir `@Query`.
- **Optimistic Locking con @Version:** Se implementó `@Version` tanto en `Reservation` como en `Lodging`. La version solo en `Reservation` no previene inserts concurrentes, ya que las entidades nuevas arrancan con `version=0`. La version en `Lodging` provee un punto de contención único.
- **ConsoleEmailService:** Se modificó para loggear en consola en lugar de enviar emails reales, evitando consumir el límite gratuito de Mailtrap durante el desarrollo.
- **DTOs en Subcarpetas por Dominio:** Se reorganizó la estructura de `dto/` con subcarpetas por dominio (`lodging/`, `reservation/`, `auth/`, `category/`, `features/`, `user/`), mejorando la navegabilidad y escalabilidad.
- **Secretos en Variables de Entorno:** Todos los secrets (JWT, Cloudinary, BD) se movieron a variables de entorno con `.env.local` en `.gitignore`, y `.env.example` con placeholders para el repositorio.

- **SpringDoc OpenAPI:** Se incorporó documentación automática de la API con Swagger UI en `/swagger-ui/index.html` y configuración personalizada con título y descripción del proyecto.

7. Testing

- **120 tests backend:** Todos verdes. Incluyen unitarios (Mockito), integración (MockMvc + Testcontainers con MariaDB 10.11), y pruebas de mapeo de entidades.
- **Cobertura:** CRUD de todas las nuevas entidades, búsqueda con filtros, solapamiento de fechas, seguridad de endpoints, y validación de DTOs.
- **Frontend:** Sin test runner configurado. Validación manual (smoke tests).

8. Limitaciones Conocidas y Deuda Técnica Controlada

1. **Refresh Tokens:** El sistema carece de refresh tokens. El JWT expira a las 8 horas forzando reautenticación. Pendiente para futura iteración.
2. **Precios por Temporada:** El precio por noche es fijo (`pricePerNight` en Lodging). No hay soporte para tarifas variables por temporada.
3. **Subida de Imágenes:** La integración con Cloudinary está implementada pero la UI de subida en el panel admin no está conectada al frontend.
4. **Filtro por Features en Búsqueda:** El endpoint `findAvailable` acepta `featureIds` pero el frontend de SearchResults no lo expone como filtro.
5. **Notificaciones por Email:** Desactivadas en desarrollo (ConsoleEmailService en modo log). Requieren reactivar Mailtrap o configurar SMTP real para producción.
6. **Frontend sin Tests Automatizados:** No hay test runner configurado en el frontend. Las validaciones son manuales.